

정규식 활용한 데이터 검증

01 명령어에서 정규식 활용

02 프로그램에서 정규식 활용



학습 목표

- ✓ 리눅스 시스템에서 정규식을 활용하여 정보를 처리한다.
- ✓ 자바 프로그램에서 정규식을 활용하여 정보를 처리한다.



학습 내용

1. 리눅스 터미널에서 활용할 수 있는 정규식 활용법
2. 자바 프로그램에서 활용할 수 있는 정규식 활용법



정규식 활용한 데이터 검증

01 명령어에서 정규식 활용

02 프로그램에서 정규식 활용

grep 명령어에서 정규식 사용

grep 명령어

grep(Global Regular Expression Print)은 리눅스에서 문자열을 검색하는 강력한 명령어. 특정 패턴을 파일이나 표준 입력에서 찾아 출력하는 역할을 하며, 정규 표현식을 지원하여 다양한 패턴 매칭을 수행할 수 있다.

grep 명령어 기본 사용법

grep [옵션] "검색할 패턴" 파일명

\$ grep "hello" sample.txt ← sample.txt 파일에서 hello가 포함된 모든 라인을 출력

grep 명령어에서 정규식 사용

grep 명령어에서 자주 사용하는 옵션

옵션	설명
-i	대소문자 구분 없이 검색
-v	패턴과 일치하지 않는 줄 출력
-n	검색된 줄의 줄 번호 출력
-c	패턴과 일치하는 줄의 개수만 출력
-r	디렉토리 내 모든 파일을 재귀적으로 검색
-l	패턴이 포함된 파일 이름만 출력
-w	단어 단위로 정확히 일치하는 것만 검색
-E	정규 표현식 활용

grep 명령어에서 정규식 사용

정규식을 활용한 grep 명령어 활용 예

(예시 1) log.txt 파일에서 "error" 또는 "warning" 이 포함된 줄 출력

```
$ grep -E "error|warning" log.txt
```

(예시 2) number.txt 파일에서 숫자로 시작하는 줄 만 출력

```
$ grep -E "^[0-9]+" number.txt
```

(예시 3) user.txt 파일에서 이메일 주소 패턴을 찾아서 출력

```
$ grep -E "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" user.txt
```

sed 명령어에서 정규식 사용

sed 명령어

sed(Stream Editor)는 텍스트 파일을 수정하거나 필터링할 때 사용하는 강력한 스트림 편집기.
grep이 텍스트에서 특정 패턴을 찾는다면, sed는 텍스트를 수정(변경, 삭제, 추가 등)할 수 있는 도구이다.

sed 명령어 기본 사용법

sed [옵션] "명령어" 파일명

\$ sed 's/apple/orange/' fruits.txt ← fruits.txt 파일에서 첫번째 apple을 찾아서 orange로 변경해서 출력

sed 명령어에서 정규식 사용

sed 명령어에서 자주 사용하는 명령어

명령어	설명
s/패턴/변경할 문자열/	문자열 바꾸기
d	특정된 라인 삭제
p	특정된 라인 전체 출력
a\ 텍스트	특정된 라인 다음에 지정한 텍스트 줄을 추가
i\ 텍스트	특정된 라인 앞에 지정한 텍스트 줄을 추가
c\ 텍스트	특정된 라인을 지정한 텍스트 줄로 변경

sed 명령어에서 정규식 사용

(예시 1) HTML 주석문 제거

```
$ sed -E -i.bak 's/<!--.*?-->//g' index.html
```

(예시 2) IPv4 주소 마스킹

```
$ sed -E -i.bak 's/([0-9]{1,3}\.){3}[0-9]{1,3}/***.***.***.***/' access.log
```

(예시 3) 전화번호 형식 통일

```
$ sed -E -i.bak 's/\b([0-9]{3})-?([0-9]{3,4})-?([0-9]{4})\b/\1-\2-\3/g' contact.txt
```

awk 명령어에서 정규식 사용

awk 명령어

awk는 텍스트 데이터 처리와 분석을 위한 강력한 도구로, 특히 열(column) 단위 데이터 처리, 조건별 출력, 연산 등을 쉽게 수행
주로 로그 분석, CSV 파일 처리, 텍스트 필터링 등에 사용된다.

awk 명령어 기본 사용법

awk '패턴 {액션}' 파일명

\$ awk '{print \$2}' data.txt ← data.txt 파일의 두번째 컬럼 출력

awk 명령어에서 정규식 사용

awk의 주요 변수

변수	의미	설명
\$0	Current Record	현재 행 전체
\$1, \$2, ...	Specific Record	첫번째, 두번째, .. 컬럼값
NR	Number of Records	현재 줄 번호 (1부터 시작)
NF	Number of Fields	현재 행의 컬럼 개수
FS	Field Separator	입력 필드 구분자 (기본값: 공백)
OFS	Output Field Separator	출력 필드 구분자 (기본값: 공백)
RS	Record Separator	입력 줄 구분자 (기본값: 개행 \n)
ORS	Output Record Separator	출력 줄 구분자 (기본값: 개행 \n)

awk 명령어에서 정규식 사용

(예시 1) 특정 도메인의 이메일 주소 추출

```
$ awk '/@example\.com$/ { print $0 }' users.txt
```

(예시 2) 숫자로만 이루어진 행을 제외하고 출력

```
$ awk '!/^([0-9]+)$/ { print $0 }' data.txt
```

(예시 3) ERROR 또는 WARN으로 시작하는 로그 줄만 추출

```
$ awk '/^(ERROR|WARN)/ { print $1, $3 }' log.txt
```



정규식 활용한 데이터 검증

01 명령어에서 정규식 활용

02 프로그램에서 정규식 활용

자바 프로그램에서 정규식 사용

정규식을 다루는 주요 클래스

- Pattern : 정규식을 받아들여 패턴을 정의
- Matcher : 입력 문자열과 정의된 패턴을 비교하여 일치 여부 검사
- String의 `.matches()`, `.replaceAll()` : 간단한 작업을 정규식으로 처리

주요 용도

- 특정 패턴이 포함된 문자열 찾기 (Pattern & Matcher)
- 입력 검증 (이메일, 전화번호, 비밀번호 등)
- 문자열 치환 및 필터링 (`.replaceAll()`)

자바 프로그램에서 정규식 사용

1. 특정 패턴이 포함된 문자열 찾기

```
import java.util.regex.*;

public class RegexExample {
    public static void main(String[] args) {
        String text = "My email is user@example.com and my backup is admin@mail.com";
        String regex = "\\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}\\b"; // 이메일 패턴

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Found: " + matcher.group());
        }
    }
}
```

자바 프로그램에서 정규식 사용

2. 사용자 입력 검증

```
public class RegexValidation {  
    public static void main(String[] args) {  
        String email = "user@example.com";  
        String phone = "010-1234-5678";  
  
        System.out.println(email.matches("^[\\w.%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$")); // 이메일 검증  
        System.out.println(phone.matches("^\\d{3}-\\d{3,4}-\\d{4}$")); // 전화번호 검증  
    }  
}
```

자바 프로그램에서 정규식 사용

3. 문자열 치환

```
public class RegexReplace {  
    public static void main(String[] args) {  
        String text = "My password is secret123";  
        String censored = text.replaceAll("password is \\w+", "password is ****");  
  
        System.out.println(censored);  
    }  
}
```

자바 프로그램에서 정규식 사용

4. 비밀번호 강도 검사 (최소 8자, 대소문자, 숫자 포함)

```
import java.util.regex.*;

public class PasswordRegexTest {
    public static void main(String[] args) {
        // 정규식 패턴
        String passwordRegex = "^(?=.*[A-Z])(?=.*\\d)(?=.*[a-z]).{8,}$";
        Pattern pattern = Pattern.compile(passwordRegex);

        // 테스트할 비밀번호 목록
        String[] testPasswords = {
            "Password123", // 유효함
            "password",    // 대문자, 숫자 없음
            "PASSWORD123", // 소문자 없음
            "Pass12",      // 8자 미만
            "SecureP@ssword1" // 유효함
        };

        // 정규식 테스트
        for (String password : testPasswords) {
            Matcher matcher = pattern.matcher(password);
            boolean isValid = matcher.matches();
            System.out.println("패스워드: " + password + " → " + (isValid ? "유효함 ☑" : "유효하지 않음 ✕"));
        }
    }
}
```

정규식 사용 시 문제점

1. 성능 문제

- 정규식은 백트래킹(Backtracking)을 사용하여 패턴을 찾기 때문에 복잡한 정규식일수록 실행 속도가 느려 질 수 있음.
- 특히 비효율적인 패턴(ex: .* 사용, 중첩된 그룹 등)은 지수적 시간 복잡도를 초래할 수 있음.

(비효율적인 정규식 예)

```
$ grep -E "(a+)+b" largefile.txt
```

a+가 여러 번 반복되면서 백트래킹이 많아져 속도가 느려짐.

정규식 사용 시 문제점

2. 과도한 매칭(Overmatching)

- 정규식이 너무 많은 결과를 매칭하는 경우가 있음
- 특히 .* 를 사용하면 예기치 않은 부분까지 매칭될 수 있음.

(과도한 매칭 예)

```
$ echo "<title>Example</title>" | grep -o "<.*>"
```

<title>태그 하나만 찾으려 했지만 모든 태그를 포함하는 부분을 찾음

정규식 사용 시 문제점

3. 예상치 못한 미매칭(Undermatching)

- 정규식이 특정한 경우를 잡아내지 못하는 경우가 있음
- 예를 들어, 공백, 특수문자, 대소문자 변형 등을 고려하지 않으면 중요한 데이터를 놓칠 수 있음

(미매칭 예)

```
$ grep -E "http://" urls.txt
```

https://example.com 은 매칭 되지 않음 "https?:://" 정규식을 사용하면 http:// 와 https:// 모두 매칭 가능

정규식 사용 시 문제점

4. 정규식 가독성 문제(Readability Issues)

- 정규식이 길어지면 읽기 어렵고 유지보수가 힘들어짐.

(가독성이 낮은 정규식 예)

```
Wb[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+W.[A-Za-z]{2,}Wb
```

이메일 주소를 찾는 정규식이지만 가독성이 떨어짐. 설명 주석을 추가하거나 구조적 표현을 사용해서 가독성을 개선할 수 있음

정규식 사용 시 문제점

5. 보안문제(ReDoS, Regular Expression Denial of Service)

- 일부 정규식은 특정 입력에 대해 매우 느리게 동작하여 서비스거부(DoS) 공격을 유발 할 수 있음
- 특히 .*, .+, (a|b|c|d|e|f)* 같은 패턴이 문제를 일으킬 수 있음.

(ReDoS 발생 가능 예)

```
echo "aaaaaaaaaaaaaaaaaaaaaaaX" | grep -E "^(a+)+X$"
```

(a+)+ 패턴은 백트래킹이 많아져 실행 시간이 길어짐.



요약 정리

- 자바에서 정규식을 활용하는 주요 클래스로 Pattern, Matcher가 있다.
- 정규식의 패턴 매칭을 통해 문자열 검색 및 입력 검증이 가능하다.
- `replaceAll()`을 이용하여 문자열 치환이 가능하다.



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Several network-like structures, consisting of white dots connected by thin white lines, are scattered across the image. There are also solid blue rectangular bars of different sizes and orientations. The overall aesthetic is modern and technological.

감사합니다.



퀴즈

1. 다음 리눅스 명령어 중 정규식을 활용할 수 없는 것은?

1) grep

2) sed

3) ps

4) awk

2. 자바 프로그램에서 정규식을 받아들여 패턴을 정의하는데 사용하는 클래스명은?

()

3. 다음 중 정규식 사용시 발생 가능한 문제점이 아닌것은?

1) DDoS 보안문제

2) 미매칭 문제

3) 성능 문제

4) 과도한 매칭



퀴즈 정답

1. 다음 리눅스 명령어 중 정규식을 활용할 수 없는 것은?

3) ps

ps는 프로세스의 리스트를 보는 명령어이며, 자체적으로는 정규식을 지원하지 않습니다.

2. 자바 프로그램에서 정규식을 받아들여 패턴을 정의하는데 사용하는 클래스명은?

(Pattern)

3. 다음 중 정규식 성능 저하의 원인이 아닌 것은?

1) DDoS 보안 문제

DDoS는 분산형 서비스 거부이며, 정규식으로 발생할 수 있는 보안 문제는 수많은 백트래킹으로 인한 성능저하를 일으키는 reDoS 보안 문제임.